

CPRE 288 Final Project Report

ProtoCue

Project Group 4 - Members

Adrian Thongsavath

Deraj Balamurugan

Lex Marshall

Noah Gratz

Shawn Robinson

Shun Quinlan

Iowa State University

Dr Phillip Jones

December 2025





Table of Contents

Table of Contents.....	1
1. Introduction.....	3
2. Problem Statement.....	4
3. User Research & Design Foundations.....	5
3.1 Empathy Map + POV.....	5
3.1.2 Combat Commander.....	5
3.1.3 Wounded Soldier.....	6
3.1.4 Support Medics.....	7
3.2 Research Points.....	8
4. System Design & Architecture.....	10
4.1 System Architecture Overview.....	10
4.1.1 Movement Subsystem.....	11
4.1.2 Detection Subsystem.....	11
4.1.3 Orientation Subsystem.....	12
4.1.4 Communication Subsystem.....	12
4.1.5 Feedback & Output Subsystem.....	13
4.1.6 Architectural Design Principles.....	13
4.2 Hardware Interface Summary.....	14
4.3 External Hardware Integration.....	15
4.4 Mapped Functionality Diagram.....	17
4.5 Field Operation Logic.....	19
4.6 Graphic User Interface (GUI).....	22
4.6.1 GUI Purpose.....	23
4.6.2 Communication Architecture.....	24
4.6.3 Real-Time Visualization.....	26
4.6.4 Logging & Diagnostics.....	26
4.6.5 Role in Blind Navigation.....	29
5. System Operation & Testing.....	30
5.1 User Sketch.....	30
5.2 Base Functionality.....	31
5.3 CyBot Capabilities.....	33
5.4 Mapping Test Field Elements to Application Narrative.....	34
5.4 Test Field Sketches.....	35
5.5 Narrative of Testing Experiences.....	37
6. Performance Evaluation.....	38
6.1 Mission Completion Performance.....	38



6.2 Hazard Detection & Response.....	38
6.3 Target Identification Performance.....	39
6.4 System Stability.....	39
6.5 System Stability.....	40
7. Engineering Challenges & Solutions.....	41
7.1 Target Identification Strategy Evaluation.....	41
7.2 Phase 1 – LDR + IR Calibration.....	41
7.3 Phase 2 – Transition to IR Modulation.....	42
7.4 Phase 3 – Enclosed IR Modulation Housing.....	42
8. Future Improvements.....	43
9. Team Contributions.....	45
10. Conclusion.....	47
Works Cited.....	48



1. Introduction

Modern combat environments present extreme challenges for casualty recovery operations. Injured personnel may be located in unstable terrain, obstructed by debris, or positioned within active threat zones where direct human intervention introduces significant risk. Rapid identification and safe access to wounded soldiers are critical factors in reducing preventable loss of life.

Advancements in embedded systems and autonomous robotics create opportunities to reduce this risk by deploying remote reconnaissance platforms into hazardous environments. ProtoCue was developed as an embedded rescue-scout system designed to navigate complex terrain, detect friendly distress signals, and support safer casualty localization before human medics are deployed.

By integrating real-time sensor processing, hazard detection, and directional signaling, ProtoCue demonstrates how embedded control systems can extend operational reach while minimizing direct human exposure to danger.



2. Problem Statement

In battlefield scenarios, injured soldiers may be located in environments that are unsafe for immediate human retrieval due to obstacles, debris, or ongoing threats. Traditional rescue operations require personnel to enter unknown terrain without full situational awareness, increasing the risk of secondary casualties.

The objective of ProtoCue is to design and implement an embedded system capable of:

- Navigating from any edge of a designated field environment
- Avoiding physical obstacles and hazardous drop-offs
- Detecting a friendly distress signal using infrared-based identification
- Providing real-time feedback to operators
- Locating the target within a defined time constraint (20 minutes)
- Exiting the hazardous zone after identification

The system must operate reliably under varied field configurations and environmental conditions while maintaining stable real-time execution and sensor integration.



3. User Research & Design Foundations

3.1 Empathy Map + POV

3.1.2 Combat Commander





3.1.3 Wounded Soldier





3.1.4 Support Medics





3.2 Research Points

- According to the MHS (Military Health System), the leading cause of hospital visits for the combined U.S. military in 2024 was injury -- far greater than any physical or mental illness visits [3]. (See figure 1a, borrowed from their website)
 - Over 500,000 individual service members were treated for injuries in hospitals in 2024, resulting in over 3.2 million visits.
 - Clearly, not only is there a moral and patriotic element to protecting soldiers after combat injuries, but there is also a significant systemic and financial argument for this technology investment. Faster scouting and retrieval of injured soldiers will reduce the time it takes for their injuries to worsen, thereby reducing the total treatment required and, consequently, the burden on the military medical system.
- There is a meaningful distinction between CASEVAC and MEDEVAC in US Military operations. The central distinction seems to be formality and urgency. CASEVAC is more haphazard and may use non-traditional medical vehicles. MEDEVAC is a pre-planned, fully equipped mobile medical unit. CASEVAC is often utilized when a soldier's condition is too severe to wait for MEDEVAC [2].
 - ProtoCue can be designed to assist in both CASEVAC and MEDEVAC instances [2].
 - Primarily, ProtoCue eliminates the need for human soldiers to risk their lives to find and retrieve soldiers in an active combat zone. What's more, ProtoCue's array of sensors can detect soldiers from vantage points and perspectives that would be difficult for the human eye, especially in a stressful, time-sensitive environment for the human scout. Robots don't get nervous, and they can continue forward even when a human should rightfully take cover from a near-miss [2].
 - Upon finding the soldier, opportunities for more robust MEDEVAC are made more convenient with ProtoCue, which can send a direct beacon (output



information) to an air support vehicle to assist in retrieving the wounded individual. Between MEDEVAC's arrival and ProtoCue's signal, ProtoCue's "found_soldier" protocol is modal and flexible enough to add auxiliary functionality, such as dispensing pre-approved medical equipment to the soldier. Perhaps a future iteration of the prototype could even take a literal "hands-on" approach to delivering care to soldiers. [2]

- This research paper supports the assertion that automated MEDEVAC, particularly in a distributed system of autonomous vehicles, greatly reduces the time per casualty incident to patient treatment [1]
 - This supports the conclusion that the greater investment in autonomous solutions like ProtoCue is made, the greater return our service people will see in the field [1]



4. System Design & Architecture

4.1 System Architecture Overview

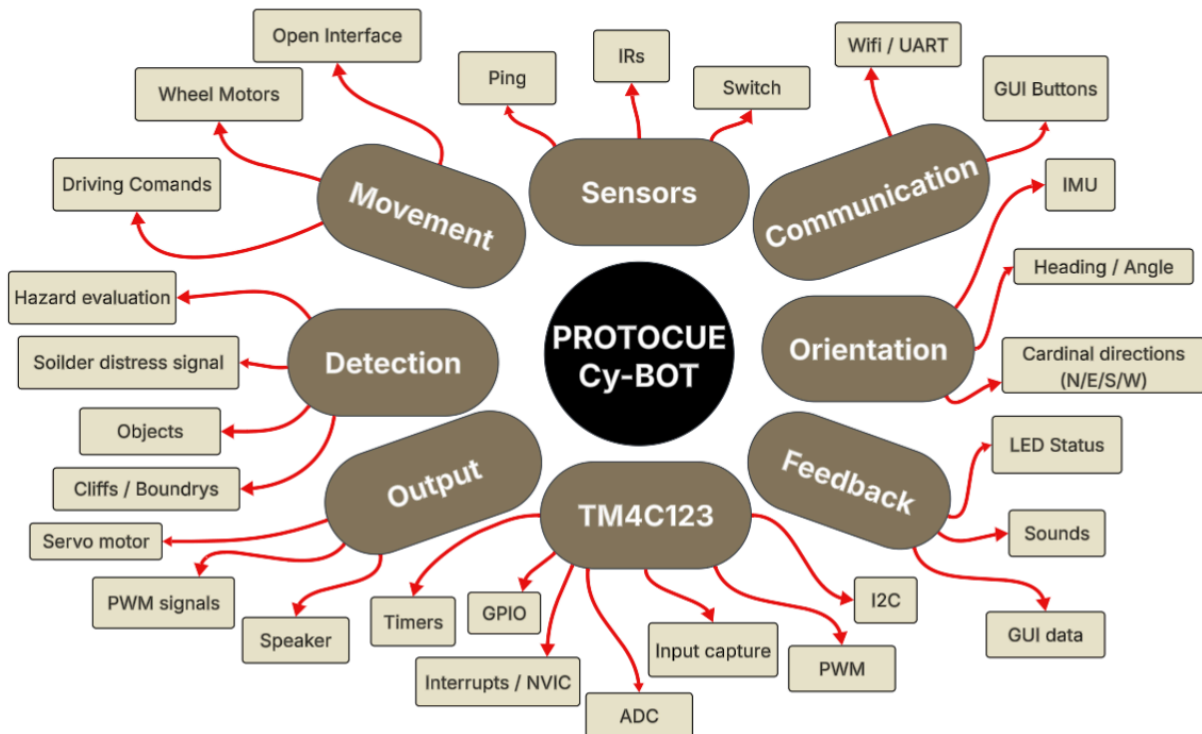


Figure 1: The diagram groups ProtoCue Cy-Bot's main capabilities into the following areas: movement, sensing, communication, detection, orientation, feedback, output devices, and MCU support. Together, these petals show how the microcontroller features, sensors, and interfaces combine to enable the wounded soldier rescue mission.

ProtoCue's system architecture is organized around a modular, layered control structure centered on the TM4C123GH6PM microcontroller. As illustrated in Figure 1, the system is divided into coordinated subsystems responsible for movement, sensing, detection, communication, orientation, feedback, and output control. Each subsystem operates with defined responsibilities and communicates via the microcontroller's hardware interfaces (GPIO, ADC, PWM, I2C,



UART, and interrupts/NVIC). This modular organization ensures clear separation of functions while maintaining centralized control, allowing the system to remain scalable, maintainable, and reliable during mission operation.

4.1.1 Movement Subsystem

The Movement subsystem controls wheel motors and servo positioning to enable navigation through the field environment. Motion commands originate from either the driver via the GUI or internal logic during scanning routines.

The subsystem leverages the following hardware interfaces and control strategies:

- PWM for motor control
- Open Interface for wheel actuation
- IMU-based heading feedback for directional correction

This allows ProtoCue to execute controlled turns, scanning rotations, hazard avoidance maneuvers, and forward traversal toward the detected target.

4.1.2 Detection Subsystem

The detection subsystem is responsible for environmental awareness and target identification. It integrates:

- IR receiver and 38 kHz modulated IR transmitter system
- PING ultrasonic sensor for distance measurement
- Bumper sensors for collision detection
- Cliff sensors for boundary detection



Sensor data is processed by the microcontroller using interrupt-driven logic and timed sampling routines. The IR modulation strategy enables selective detection of the designated distress signal while minimizing environmental interference.

4.1.3 Orientation Subsystem

The orientation subsystem provides directional awareness using the BNO055 IMU via I2C communication.

This subsystem:

- Tracks heading (cardinal directions)
- Supports driver orientation through GUI feedback
- Enables consistent navigation relative to the starting position

Orientation data improves scanning precision and helps drivers interpret field position during manual control in blind areas.

4.1.4 Communication Subsystem

The Communication subsystem links ProtoCue to the external Graphical User Interface (GUI) through WiFi-to-UART communication.

This subsystem:

- Receives driver commands (movement, scan, mission start)
- Transmits sensor data summaries
- Sends hazard alerts and mission status updates
- Logs movement and scan data for visualization

Bidirectional communication ensures the remote operator has real-time situational awareness.



4.1.5 Feedback & Output Subsystem

The feedback subsystem communicates the mission state both locally and remotely. It includes:

- Status LEDs (goal detected, hazard detected)
- Audio signals for mission confirmation
- GUI graphical mapping and scan visualization

This layered feedback approach improves operator decision-making under time constraints.

4.1.6 Architectural Design Principles

The architecture follows a modular and interrupt-driven design, ensuring:

- Deterministic timing behavior
- Separation between sensing, decision-making, and actuation
- Scalable integration of additional hardware
- Reliable performance under live field testing conditions

Each subsystem operates independently but is coordinated centrally through the microcontroller's hardware abstraction and state-machine logic. This structure allowed ProtoCue to maintain stability, real-time responsiveness, and reliable mission execution across varied field layouts.



4.2 Hardware Interface Summary

The external components were integrated using a combination of digital, analog, and communication protocols supported by the TM4C123 microcontroller. Each component was selected and positioned to maximize sensing accuracy while maintaining electrical stability and mechanical balance.

Component	Interface	Function Within System
BNO055 Inertial Measurement Unit (IMU)	I2C	Provided cardinal orientation and heading information for directional control.
Espressif (ESP) 32-C6	UART (Serial Communication) / WIFI	Provided wireless communication and serial data exchange between the embedded controller and external systems (GUI interface), enabling remote monitoring and control.
TM4C123GH6PM Microcontroller	GPIO, I2C, ADC, PWM, UART, timer interrupts, and open interface.	Acted as the primary real-time processor, responsible for motor control, sensor data acquisition (IMU, PING, bump, cliff), IR modulation, and interrupt-based task scheduling.
Infrared (IR) Modulator	GPIO / PWM	Generated a modulated IR signal for goal identification.
IR Transmitters (PVC)	General Purpose Input/Output (GPIO)	Emitted identification signals for soldier detection.
PING Ultrasonic Sensor	Digital Trigger / Echo	Measured distance to obstacles and field boundaries.
Status LEDs	GPIO	Indicated system state (goal detected, hazard detected, mission complete).
Bumper Sensors	Digital Input	Detect physical collisions with obstacles to prevent forward motion into hazards.
Cliff Sensors	ADC	Identified hazardous drop-offs in terrain.

4.3 External Hardware Integration

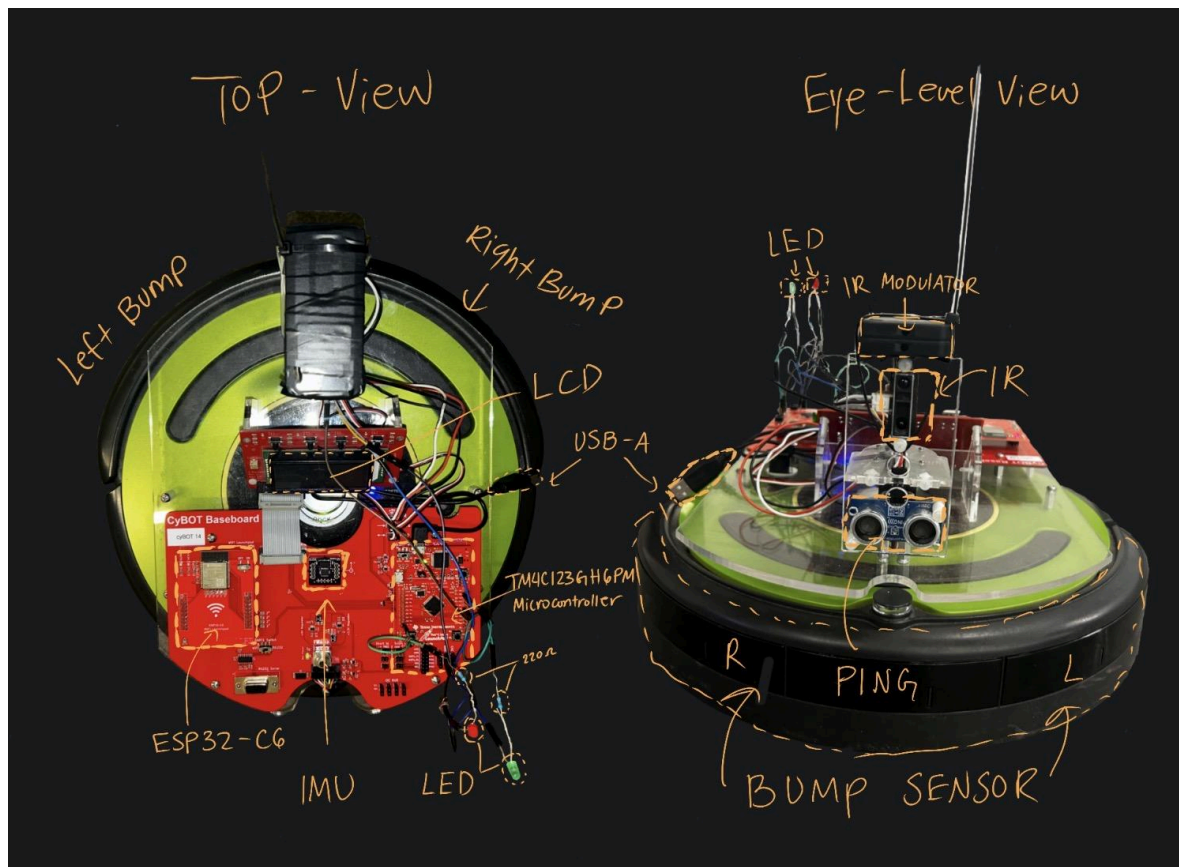


Figure 2: This illustrates the physical integration of external hardware components mounted onto the CyBot platform. The left image (top view) highlights internal wiring and board-level connections, while the right image (eye-level view) shows the forward-facing sensor configuration used during field operation.

The top-view configuration highlights board-level integration, including connections to the TM4C123 microcontroller, the BNO055 IMU module, the ESP32 WiFi module, status LEDs, and IR modulation circuitry. GPIO, I2C, UART, and power lines were routed to reduce interference while maintaining accessible debugging points.

The forward-facing sensor layout was optimized for environmental awareness and target detection. The PING ultrasonic sensor was mounted centrally to provide direct distance measurement ahead of the robot. IR transmitters and the 38 kHz IR modulator were positioned to maximize directional signal detection, while a narrowed housing enclosure was implemented to reduce false positives caused by wide detection cones.



Bumper sensors and cliff sensors remained accessible for real-time hazard detection. Their placement ensured immediate interrupt-triggered responses upon physical contact or boundary detection.

This integrated hardware layout provided stable sensor performance, improved directional accuracy, and reliable communication during live mission testing.



4.4 Mapped Functionality Diagram

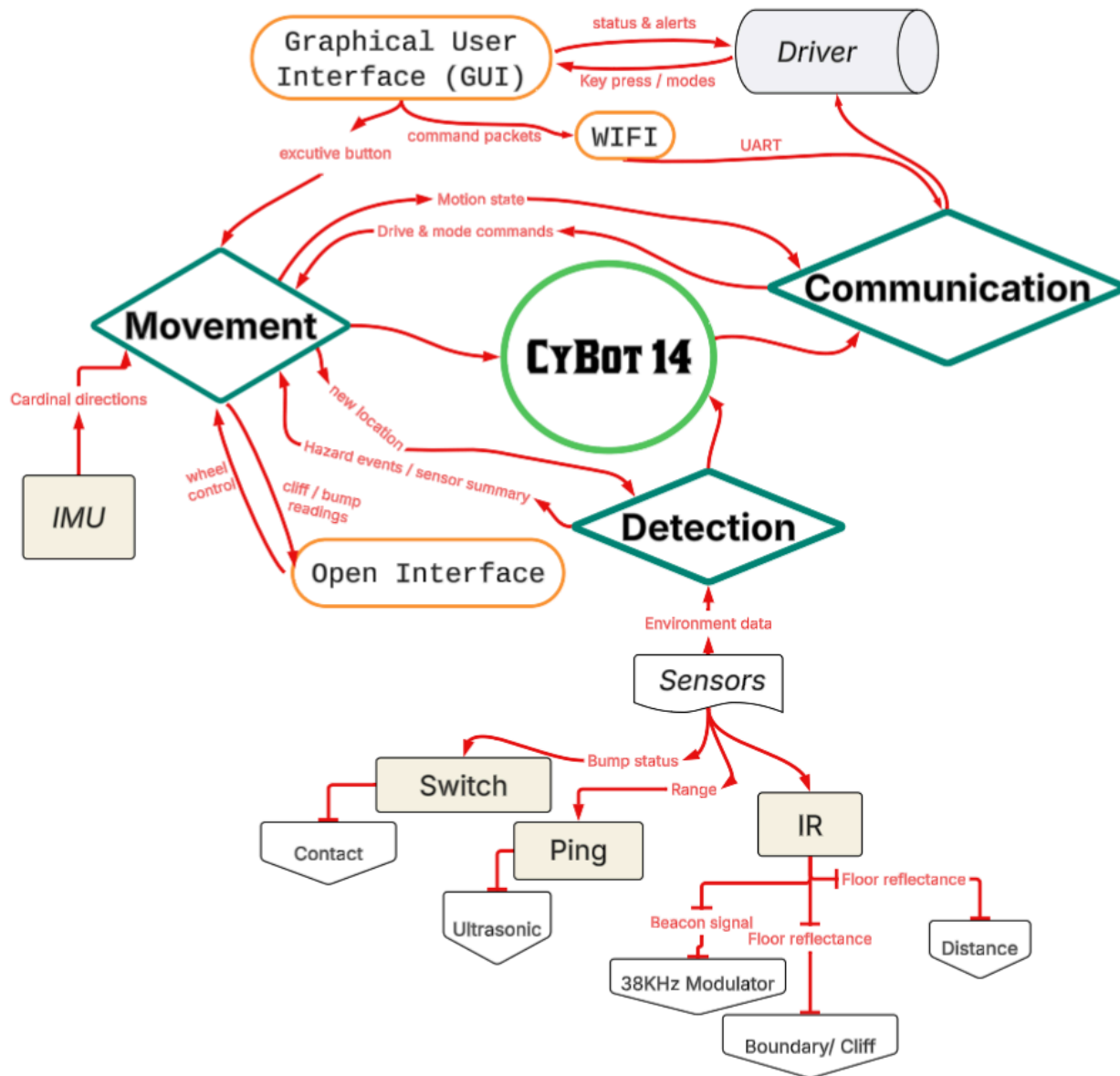


Figure 3: This diagram illustrates how CyBot (14) receives driver commands through the GUI and WiFi, utilizes its movement and detection subsystems in conjunction with the IMU, open interface, and external sensors to navigate and maintain safety, and then sends status and hazard information back to the driver via the communication subsystem.



The Mapped Functionality Diagram models the logical data flow between the operator, communication interface, microcontroller, and sensing subsystems. Driver commands originate at the GUI and are transmitted through the WiFi-to-UART interface to the TM4C123 microcontroller. These command packets include motion requests, scanning operations, and mode transitions.

Once received, the microcontroller coordinates the movement and detection subsystems. Movement control is executed via the Open Interface using wheel-motor commands and IMU-based heading correction. Simultaneously, sensor data from the IR system, the ultrasonic PING sensor, the bumper switches, and the cliff detectors are fed into the detection subsystem. This environmental data is processed to generate hazard evaluations, positional updates, and target identification signals.

Processed information is then transmitted back through the communication subsystem to the GUI as telemetry, hazard alerts, and mission status updates. This bidirectional structure forms a closed-loop control system, ensuring real-time responsiveness between operator input, autonomous sensing, and mechanical actuation.



4.5 Field Operation Logic

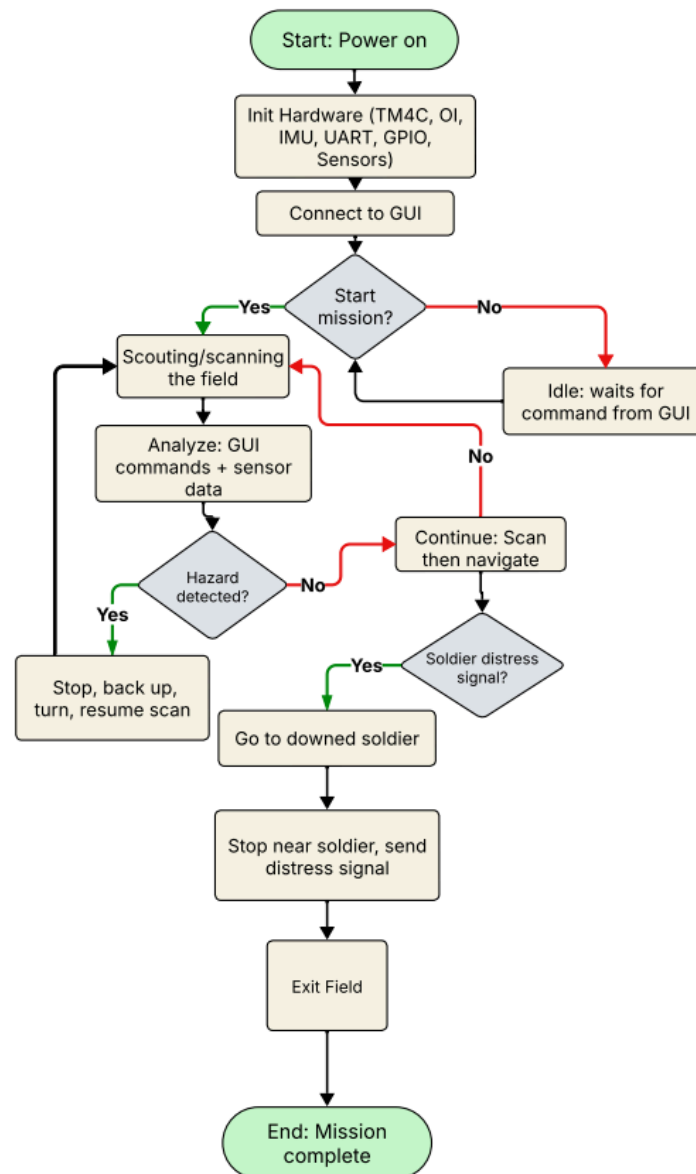


Figure 4: This diagram shows how ProtoCue operates in the field, from power-on and hardware initialization through connection to the GUI, scouting, and continuous analysis of sensor data. Hazards (cliffs, bumps, and obstacles) are handled with a safety routine that stops, backs up, and reports the event, while a detected soldier distress signal triggers an approach, an alarm, and a GUI notification. After the soldier is found, the driver can command ProtoCue to exit the field. At this point, the robot manually navigates to the nearest exit, and the mission is completed.



The Field Operation Logic diagram represents the high-level state machine governing ProtoCue's mission behavior. It defines the sequence of operational states from system initialization through mission completion, while highlighting interrupt-driven safety handling and closed-loop feedback with the GUI.

Upon power-on, the system initializes all hardware interfaces, including GPIO, UART communication, IMU (I2C), Open Interface motor control, and sensor drivers for IR, PING ultrasonic, bumper, and cliff detection. Interrupt configurations are established to prioritize hazard detection. After successfully connecting to the GUI via the WiFi-to-UART interface, the system enters an idle state and waits for a mission-start command from the operator.

Once activated, ProtoCue enters the scouting state. During this phase, the robot:

- Performs controlled rotational scanning
- Continuously processes IR and PING sensor data
- Monitors bumper and cliff interrupts
- Transmits telemetry updates to the GUI

Sensor data is analyzed in real time to evaluate hazards and detect valid distress signals.

If a hazard (cliff, bump, or obstacle) is detected, interrupt-driven routines immediately override forward motion. The robot stops, backs up, adjusts heading, and resumes scanning. This ensures safety-critical events take priority over mission progression.

If a valid distress signal is identified, the system transitions into a target-approach state. The robot moves toward the detected signal while continuously verifying heading using IMU feedback and distance using ultrasonic measurements.

Upon successful localization of the downed soldier, the system triggers:

- Visual LED confirmation
- Audio signal output
- GUI mission notification



After confirmation, ProtoCue transitions into an exit state. The operator may manually navigate the robot toward a safe boundary or issue an exit command. The mission concludes once the robot leaves the field area.

This state-driven architecture ensures deterministic execution, prioritizes safety interrupts, and enables stable mission behavior across varied field conditions.



4.6 Graphic User Interface (GUI)

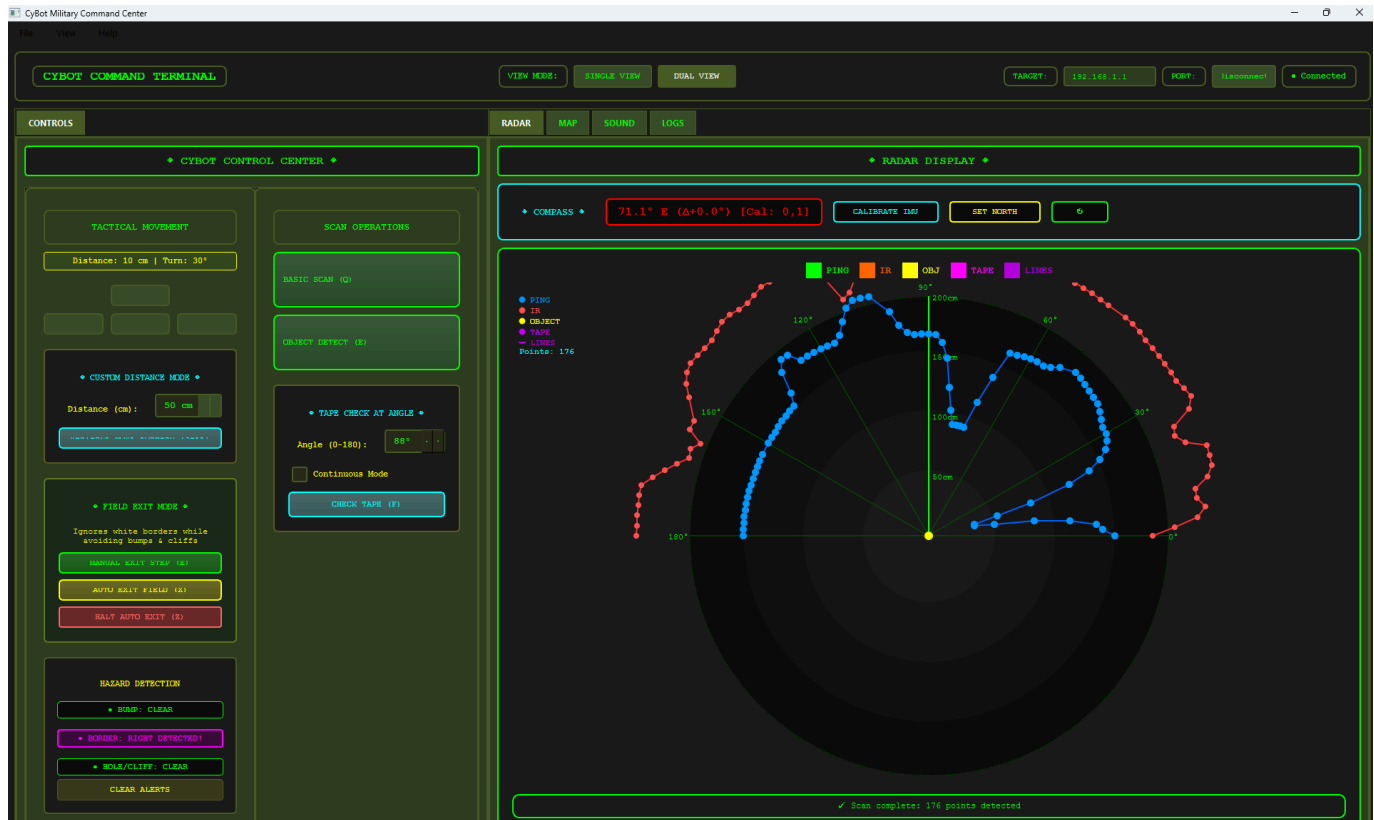


Figure 5: The ProtoCue control interface displaying real-time sensor visualization and mission control elements. The radar-style display plots IR and PING scan results relative to the robot's heading, while the left control panel provides manual movement commands, scan configuration, and hazard response controls. The interface also displays IMU heading data and system status indicators, enabling the driver to interpret environmental conditions and make informed navigation decisions.



4.6.1 GUI Purpose

The Graphic User Interface (GUI) served as the primary human–robot interaction layer for ProtoCue. While the embedded microcontroller handled real-time sensing, movement, and interrupt-driven control, the GUI provided the operator with mission-level visibility and command authority.

The GUI served four primary operational functions:

- Enable remote driver control of the CyBot platform
- Display processed sensor data in an interpretable format
- Provide real-time feedback regarding hazards and target detection
- Log mission data for performance evaluation

Rather than simply acting as a control panel, the GUI functioned as an operational dashboard. It translated raw UART telemetry into structured visual elements that supported fast decision-making under time constraints. This design enabled drivers to operate ProtoCue in “blind” areas of the field where a direct line of sight was unavailable.

By abstracting low-level embedded data into high-level visual feedback, the GUI enhanced the operator’s situational awareness.



4.6.2 Communication Architecture

The GUI communicated with ProtoCue via a WiFi-to-UART bridge implemented with an ESP32 module. The TM4C123 microcontroller transmitted structured telemetry packets over UART, which were then forwarded wirelessly to the Python-based GUI.

Communication was bidirectional:

From GUI to Robot:

- Movement commands (forward, reverse, turn)
- Scan initiation
- Mission start/stop commands
- Sound activation
- Exit-field command

From Robot to GUI:

- IR scan results
- PING distance measurements
- Hazard alerts (cliff, bump)
- IMU heading data
- Mission state indicators

The embedded system used interrupt-driven scheduling and prioritized task handling to ensure that communication did not interfere with time-critical sensing and actuation tasks. Telemetry packets were formatted consistently to maintain parsing reliability on the GUI side.

Stable UART communication was maintained throughout live testing, with no packet corruption or communication resets observed during final demonstration trials.



This communication architecture ensured deterministic embedded performance while maintaining real-time remote supervision capability.



4.6.3 Real-Time Visualization

One of the most critical features of the GUI was its ability to visualize scan data in real time.

During active scanning procedures:

- IR sensor readings were plotted radially
- PING distance measurements were displayed alongside angle values
- Detected hazards were marked on the map
- The robot's heading was displayed relative to cardinal orientation

The radar-style visualization allowed drivers to quickly interpret the following:

- Object proximity
- Potential hazards
- Target directionality

As the robot moved through the field, the GUI dynamically updated its position relative to the starting location. This created a dynamically expanding positional map referenced to the robot's origin, allowing cumulative path tracking over time.

Rather than requiring the operator to interpret raw numeric telemetry, the graphical representation reduced cognitive load and improved reaction speed. This was especially important in blind navigation scenarios where the driver relied entirely on telemetry feedback.

4.6.4 Logging & Diagnostics

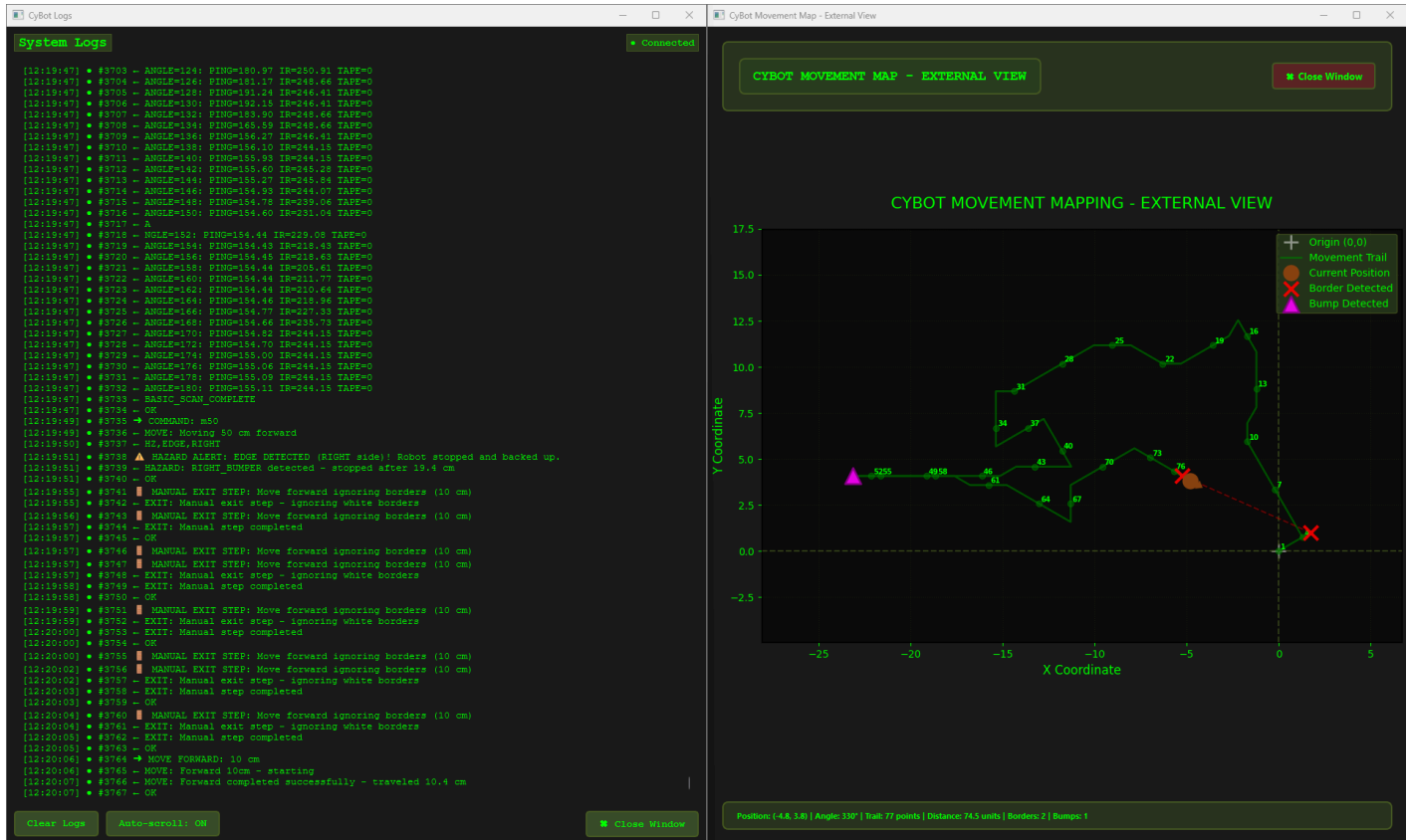


Figure 6: Real-time movement mapping and system log output during an active mission run. The left panel displays timestamped UART data, including scan values, movement commands, hazard triggers, and mission events. The right panel visualizes the robot's path relative to its starting position, including detected obstacles, boundaries, and bump events. This interface supports debugging, mission verification, and post-run performance analysis.



In addition to real-time visualization, the GUI maintained structured logs of:

- Movement commands
- Scan angle values
- IR intensity readings
- PING distances
- Hazard events
- IMU heading changes

These logs were displayed in a terminal-style window and stored for post-run analysis.

Logging served several engineering purposes:

1. Debugging sensor calibration issues
2. Identifying false positive detections
3. Evaluating hazard response timing
4. Measuring mission completion efficiency

During early test runs, logging revealed that raw IR values were difficult to interpret and that detection thresholds required refinement. These observations directly contributed to the transition from LDR-based detection to IR modulation with directional enclosure.

By maintaining persistent telemetry records, the GUI enabled data-driven system refinement rather than guess-based debugging.



4.6.5 Role in Blind Navigation

ProtoCue was frequently operated in areas where the robot was not directly visible to the driver. In these blind-navigation scenarios, the GUI became the sole source of environmental awareness.

The GUI supported blind navigation by:

- Displaying real-time obstacle distance readings
- Highlighting hazard triggers immediately upon interrupt activation
- Providing heading orientation through IMU feedback
- Visualizing scan arcs to determine safe travel paths
- Indicating successful goal detection via visual and audible alerts

Because drivers could not rely on line-of-sight control, the GUI effectively extended the robot's sensory perception to the operator. The combination of radar visualization, telemetry logs, and directional feedback enabled safe navigation even in complex field layouts.

The improved GUI interface significantly reduced operator hesitation and improved mission completion time during later testing phases.

Ultimately, the GUI elevated ProtoCue from a purely reactive sensing platform to a coordinated human-robot supervisory system.



5. System Operation & Testing

5.1 User Sketch



Figure 7: This comic-style sketch shows how ProtoCue relates to the CPRE 288 in-class demo test field. In the first panels, a remote operator manually drives and monitors the Cy-Bot via the GUI as it navigates a battlefield-style environment that mirrors our classroom test field.

ProtoCue eventually locates a wounded soldier, signals the distress event, and transmits the location back to the operator. In response, a team of medics moves in to treat and evacuate the soldier, while the robot returns toward the field base for its next mission.



5.2 Base Functionality

Base Functionality	Mapping to Application Narrative (In the form of needs statements)
Cybot Communication	For ProtoCue to be useful, it needs to maintain communication with its team at all times. Without communication, the users have no control over our robot. It needs to communicate effectively with its users at all times to perform its job.
Cybot Movement	For ProtoCue to be effective, it must navigate the playfield/battlefield safely. ProtoCue needs to take an effective route through the battlefield, minimize the number of scans, locate the wounded soldier, and escape the battlefield as quickly as possible.
Object Detection	ProtoCue needs to be aware of its surroundings. On a battlefield, your surroundings are everything. It needs to be able to see the objects that are around it. If the bot doesn't detect objects, it is going into the battlefield blind. The primary purpose of it is to gather information about where everything is; therefore, without it, the user will be unable to gather any information.
Object Avoidance	ProtoCue must avoid all objects to reach its target as soon as possible. ProtoCue needs to keep itself safe by detecting obstacles on the battlefield and avoiding them.
Boundary Adherence	ProtoCue needs to be able to scout the entire battlefield in order to locate the wounded soldier. Once ProtoCue finds them, it must vacate the premises immediately to reach safety as quickly as possible.
Arrival at Destination (Completion of Goal)	Every robot is designed to complete a goal. For ProtoCue to achieve its goal, it must locate the wounded soldier and safely and effectively escape the battlefield. ProtoCue must locate the soldier in need and escape safely in order to relay instructions.
User Interface	The effectiveness of remotely controlled and autonomous vehicles always hinges on the quality of their user



	interfaces. ProtoCue’s user interface needs to be accessible to anyone, allowing any soldier to safely save their comrade. ProtoCue’s user interface needs to display instructions to guide the user to the target soldier.
Application-Specific Functionality (opportunities for bonus)	
IR-Absorbent Tape	ProtoCue can distinguish its target by the IR-absorbing black electrical tape on all friendly uniforms. ProtoCue needs to be able to distinguish its target from its surroundings so it can relay information back to its user.
IR scan	ProtoCue requires a specialized scanner to detect the unique markings on friendly uniforms. ProtoCue requires in-depth scanning capabilities to prevent enemies from easily locating friendly soldiers on a battlefield. Friendlies don’t want to be easily scannable by enemy scans.
IMU Compass	ProtoCue needs to be able to determine its direction. By keeping track of the cardinal directions, drivers can be properly oriented to ensure they reach the soldier as quickly as possible.
Sound and LED	ProtoCue needs to give “the signal” when the wounded soldier is detected (lights up and chirps). Active soldiers on the battlefield require clear signals to determine when a task is complete. ProtoCue needs to give the wounded soldier hope that help is on the way because they don’t have access to the GUI.



5.3 CyBot Capabilities

Base Capabilities Type	Default Usage	Project Usage
Open Interface	Robot Movement	Robot Movement and cliff detection sensor
Interrupts	Ping Sensor	Ping Sensor
Wi-Fi-UART	CyBot Communications	CyBot Communications to communicate with the GUI
Analog-to-Digital Conversion	Infrared Sensor	Infrared Sensor
Input Capture	Ping Sensor	Ping Sensor
Pulse Wave Modulation (PWM)	Servo Motor	Servo Motor
Microcontroller Capabilities not Covered by Labs (opportunities for bonus)		
BNO055 IMU via I2C	Cybot's cardinal direction orientation, provided responsively, gives the drivers the exact degree the robot is facing as well as debug output on the bot's LCD.	
Custom LEDs wired into CyBot	The robot lights up green LED when a downed soldier is found and a red LED when a hazard (cliff or boundary) has been encountered	
Meaningful sound signals via an open interface	The robot has distinct sound signatures for successful initialization and after locating the goal object.	
Additional IR Modulator + IR Transmitters	A custom-taped IR Modulator (with a custom IR code) sends a higher frequency signal than the traditional IR, which then interacts with IR transmitters on our PVC to finalize goal detection	



5.4 Mapping Test Field Elements to Application Narrative

Basic Test Field Objects and other elements	Description of how Test Field Elements Map to Application Narrative
Tall wide objects	Obstacle/war debris (vague because the battlefield can be in any climate/habitat)
Short objects	Obstacle/war debris (vague because the battlefield can be in any climate/habitat)
Pillars (tall, thin objects)	Obstacle/war debris (vague because the battlefield can be in any climate/habitat)
Holes	Obstacle/war debris (vague because the battlefield can be in any climate/habitat)
Out of Bounds	Safe Zone
Destination Zone	Soldier in need
Other Application-Specific Test Field Elements (opportunities for bonus: consult with TAs and the professor)	
IR Absorbent Uniform (custom PVC-tapped black)	Soldier in need



5.4 Test Field Sketches

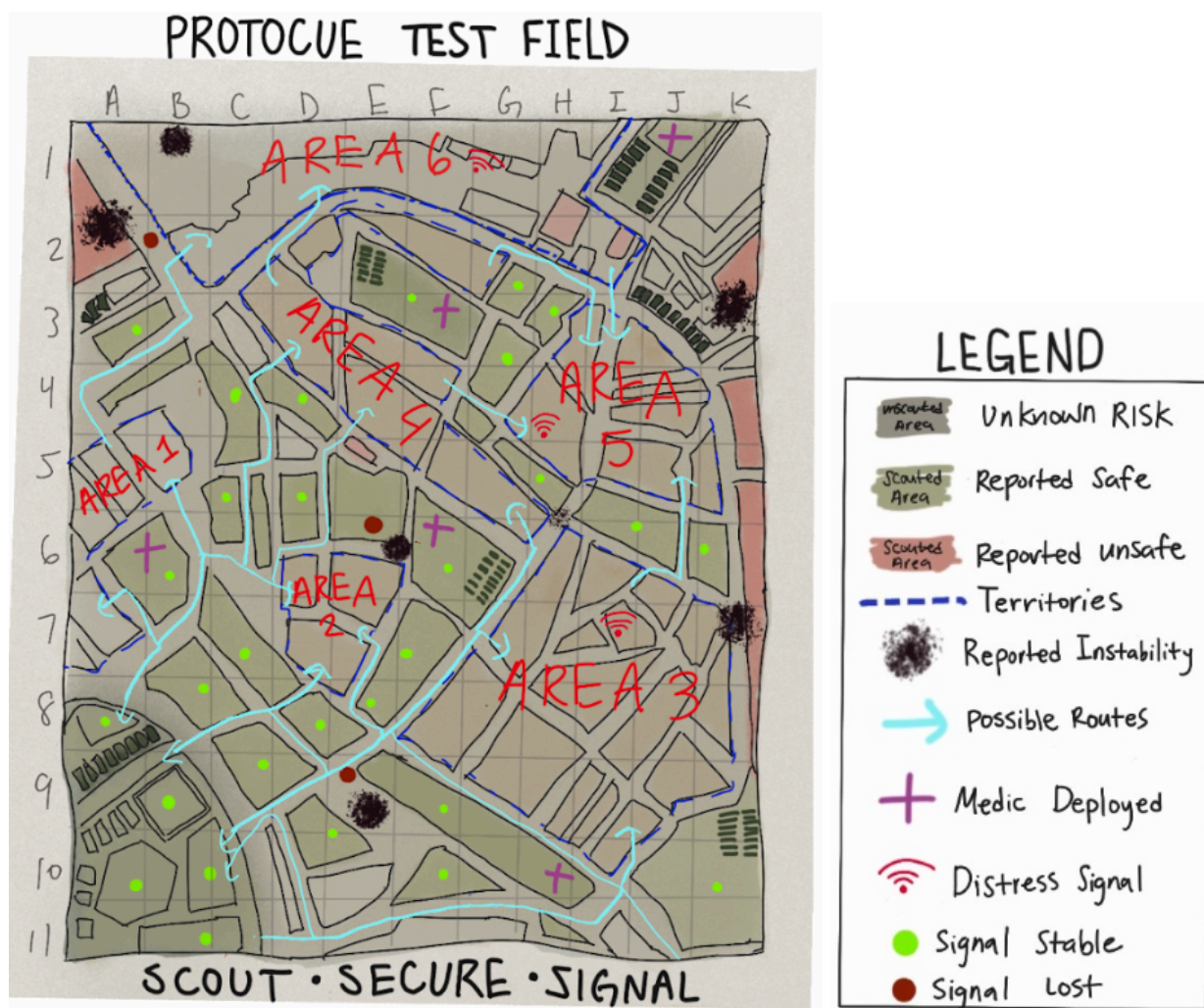


Figure 8: This sketch represents the simulated urban battlefield that ProtoCue will operate in. The field is divided into six numbered areas, with territories classified as unknown risk, reported safe, or reported unsafe based on previous scouting. The legend shows how instability reports, possible patrol routes, and medic deployment points are marked across the map. Distress-signal icons indicate downed soldiers whose signals are either stable or lost, and ProtoCue’s mission is to follow the blue routes to scout each area, secure unstable zones, and finally signal the medic to the correct location once a wounded soldier has been found.

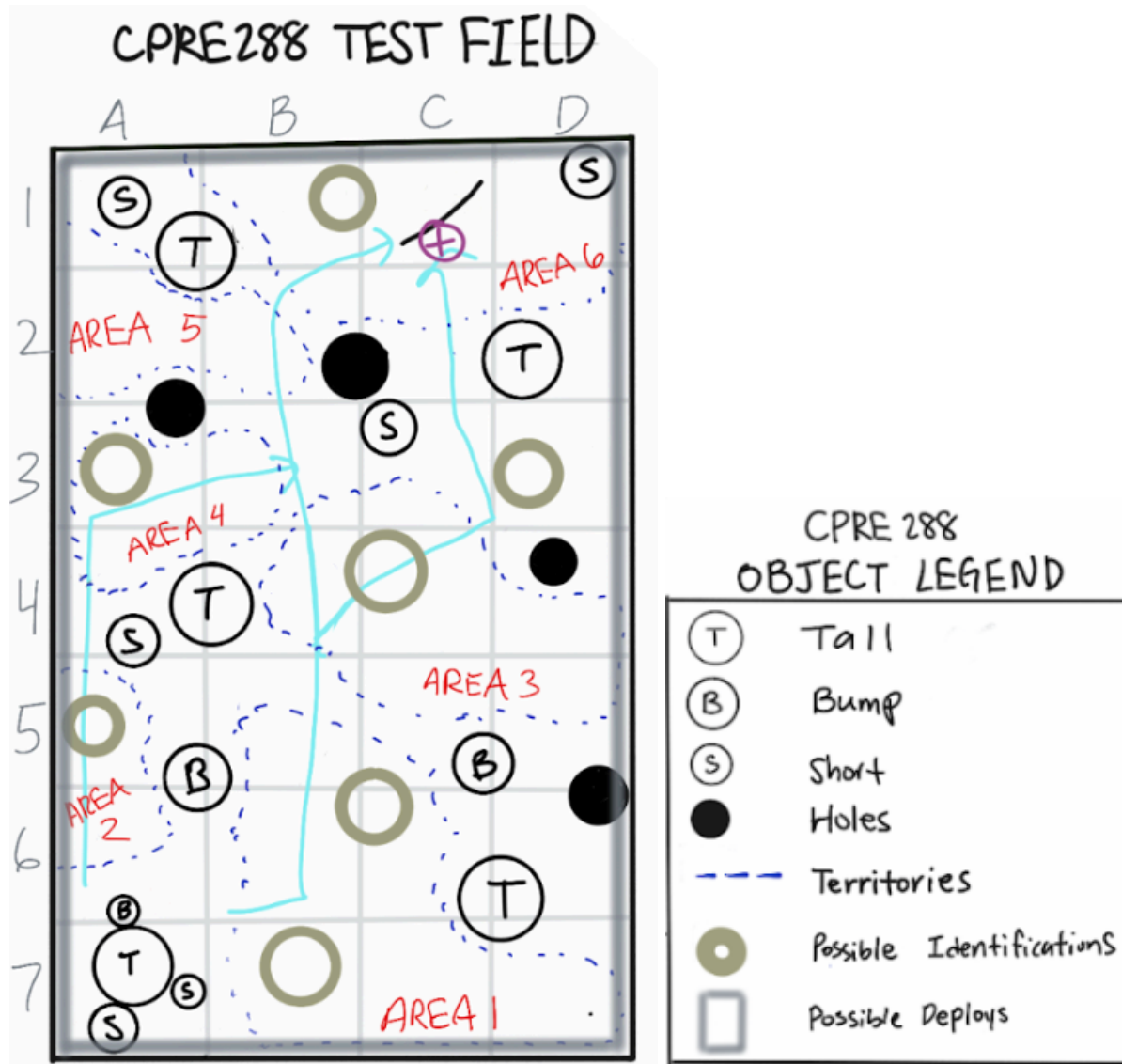


Figure 9: This diagram abstracts the battlefield scenario into the in-class demo test field used in CPRE 288. The field is divided into a grid with four columns (A–D) and seven rows (1–7). Objects include tall obstacles (T), short obstacles (S), bump posts (B), and holes, with dashed blue lines indicating territories and candidate routes through the field. The robot’s goal in the demo is to start from any point along the edge, scout through the numbered areas, identify the black “soldier” tube, and then exit the field while avoiding holes and obstacles.



5.5 Narrative of Testing Experiences

To start the testing process while integrating all of our individual classes, we threw two drivers in blind to see how far they could get. During our first test run with everything integrated, we discovered that the GUI still had some integration issues to resolve. One amusing bug we encountered was that sometimes, when we pressed the “turn left” button, ProtoCue would perform a scan. The first run revealed that many aspects were functioning properly, including ping sensor scanning, IR sensor operation, object detection, and movement.

However, there were also areas for improvement. For instance, our data wasn’t getting plotted on our chart, so every scan had to be interpreted line by line from our drivers. Moreover, the logs displayed only our raw, uncleaned IR values, so we had to look for spikes in the data, and the correct distance could not be determined. We attempted to detect our goal object using an LDR sensor and implemented a working LDR scan. However, in practice, the margin of difference in values was not significant enough to reliably find our goal during dry runs.

We implemented changes to our GUI to make sure it worked as intended in the original C code. We also conducted ongoing test runs until our drive team could consistently achieve the goal of locating the wounded soldier. We encountered a plethora of scanning errors as well. Along the way, challenges we faced included incorrectly scanning objects and misidentifying our goal object. Through intensive testing and adjustments, we mitigated the errors by continually improving our methods and logic.

Testing Process

- Assigned new features to people, where they tested them and proved them to be operating before they were merged into the main code
- Once every new feature was added, we did a simple dry run with a few objects and the wounded soldier object to see if it was implemented well
- Tweaked that feature to work better, usually before adding new features



6. Performance Evaluation

The final version of ProtoCue was evaluated based on several performance factors, including mission completion reliability, hazard detection responsiveness, target identification accuracy, and overall system stability under different field conditions. Performance was measured through a series of controlled tests as well as live demonstration trials.

6.1 Mission Completion Performance

During the final testing phases, ProtoCue was able to consistently complete its main objective of locating the target goal while safely avoiding field hazards. After several system refinements and additional driver training, the robot successfully located the target within the 20-minute mission time limit across multiple field layouts.

Across demonstration trials:

- Average mission completion time: 17.5 minutes
- Fastest recorded completion time: 4 minutes
- Successful completion rate: 1 out of 2 trials
- All successful final runs located the target within 10 minutes of active scouting

Field variability introduced additional environmental complexity, requiring drivers to interpret sensor feedback in real time. Despite differences in field layouts, the system maintained reliable detection behavior across all testing scenarios.

6.2 Hazard Detection & Response

Cliff sensors and bumper sensors were used to provide real-time hazard detection during navigation. Interrupt-driven logic allowed the system to respond immediately when a hazard was detected. During testing, the average hazard response time was approximately 3 ms, allowing the robot to quickly stop or adjust movement when obstacles were encountered. After calibration adjustments, false hazard triggers were minimal. These results demonstrate stable sensor integration and effective real-time interrupt handling.



6.3 Target Identification Performance

The refined IR modulation system significantly improved target detection reliability compared to earlier attempts using an LDR-based approach.

After implementing a directional enclosure around the IR receiver, detection accuracy improved noticeably during validation testing:

- Three consecutive successful detections during validation trials
- No inconclusive scans during the final demonstration runs
- Reduced false positives due to a narrowed detection cone
- Consistent directional feedback during scanning procedures

These improvements directly contributed to the increase in the team's final demonstration score.

6.4 System Stability

Throughout extended testing periods, the system maintained stable operation. No microcontroller resets or runtime crashes were observed during testing.

UART communication between the robot and the GUI remained stable throughout the demonstrations. Additionally, sensor polling and interrupt scheduling maintained deterministic timing behavior, allowing the robot to consistently process sensor data and respond to events without delays.

Overall, the system demonstrated stable embedded execution under live demonstration conditions.

After implementing directional enclosure refinement:

- 3 consecutive successful detections during validation testing
- No inconclusive scans during final demonstration runs
- Reduced false positives due to a narrowed detection cone
- Consistent directional feedback during scanning procedures



These improvements directly contributed to the increase in the final demonstration score.

7. Engineering Challenges & Solutions

7.1 Target Identification Strategy Evaluation

An early objective of ProtoCue was to reliably identify the target goal (a downed soldier) within a 20-minute mission time constraint. Initial detection strategies focused on using an LDR (Light Dependent Resistor) in combination with IR sensing to differentiate the target PVC pipe (marked with black electrical tape) from surrounding white field elements.



7.2 Phase 1 – LDR + IR Calibration

The system was calibrated using both IR and LDR readings to detect contrast differences between the following:

- Standard white PVC pipe
- Target PVC pipe marked with black tape

The hypothesis was that the LDR, being more sensitive to light intensity variations, would provide a measurable difference between the target and non-target surfaces.

However, after calibration and five controlled test runs:

- Raw sensor readings showed minimal differentiation
- Detection timing was inconsistent
- Target acquisition frequently exceeded the acceptable mission time
- Results were inconclusive or too slow for practical use

This approach proved unreliable under real field lighting conditions.

7.3 Phase 2 – Transition to IR Modulation

After reevaluating the detection method, the team implemented an IR modulation strategy similar to that of a television remote control. The target PVC pipe was equipped with an IR emitter serving as a “distress signal,” while the robot used an IR receiver to detect the modulated signal.

Initial tests showed promise, but the IR receiver's detection cone was too wide, leading to false positives and incorrect directional cues.



7.4 Phase 3 – Enclosed IR Modulation Housing

To improve detection precision, a physical enclosure was designed to narrow the IR sensor's field of view. This modification reduced environmental noise and improved directional accuracy during scanning.

After implementing the enclosure:

- 3 consecutive field trials successfully located the target
- All detections occurred within 10 minutes of active scouting
- False detections were significantly reduced
- Decision-making for blind manual drivers improved substantially

The refined IR modulation system ultimately provided the reliability and response time required to meet mission constraints.

8. Future Improvements

While ProtoCue successfully met its core functional requirements, several enhancements could further improve autonomy, reliability, and mission efficiency.

8.1 Autonomous Navigation & Path Planning

Future iterations could incorporate grid-based path-planning algorithms, such as breadth-first search (BFS), to enable systematic field exploration. This would reduce reliance on manual driver control and improve search efficiency under time constraints.



8.2 Return-to-Base (Go Home) Function

A dedicated “Go Home” function could be implemented to allow ProtoCue to autonomously return to its starting position after completing its mission.

This feature would require:

- Position tracking using IMU heading data and distance measurements
- Basic dead-reckoning calculations
- Stored waypoint memory
- State-machine transition logic

By recording directional movements during the scouting phase, the system could retrace its path or compute an optimized return route. This would enhance mission completeness and demonstrate higher-level autonomous control capability.

8.3 Adaptive Sensor Filtering

Although the IR modulation system improved target detection reliability, implementing dynamic thresholding or moving-average filtering would further reduce environmental noise and false detections under varying lighting conditions.

8.4 SLAM Integration

Simultaneous Localization and Mapping (SLAM) could be integrated to generate a real-time map of the field using IMU and ultrasonic sensor data.

This would allow ProtoCue to:

- Track visited areas
- Avoid redundant scanning



- Optimize search and exit routes

8.5 Hardware Optimization

Future hardware improvements may include:

- Narrower-beam IR emitters for improved directional precision
- Custom PCB integration to reduce wiring complexity
- Improved vibration isolation for the IMU
- Higher-resolution distance sensors

8.6 Enhanced Wireless Telemetry

Expanding ESP32 communication capabilities could enable real-time telemetry logging, remote diagnostics, and mission replay analysis. This would support data-driven performance evaluation and system refinement.

8.7 Vision-Based Intelligence Gathering

A forward-facing camera module could be integrated to enable real-time visual reconnaissance and environmental analysis. Unlike distance or IR-based sensing, a camera system would provide contextual field awareness, allowing the robot to:

- Identify obstacles visually
- Distinguish between field elements
- Capture images of target locations



- Transmit live video to the GUI for operator decision support

With onboard processing or external ESP32-assisted streaming, the system could evolve from reactive sensing to intelligent scene interpretation.

Future implementations may include:

- OpenMV or ESP32-CAM modules
- Basic computer vision algorithms (color detection, shape recognition)
- Object classification using lightweight machine learning models
- Snapshot-based intel logging for post-mission review

This addition would significantly enhance ProtoCue's role as a reconnaissance and intelligence-gathering platform.



9. Team Contributions

Adrian Thongsavath Primary GUI developer, converting functionality of C-based code into Python instructions and demo driver Assisted in the research and development of goal detection methods through experimental test driving Ensured success within both UX and UI for the drivers by providing the requested UI features in the final display	Deraj Balamurugan Worked on developing the Scan API and making code modular for better/repeated use. Researched and developed goal-detection methods—including those with absorbent tape, LEDs, and IR readings Provided GUI support to ensure proper data migration from C-directed functions to Python-directed displays	Lex Marshall Created and established the development environment for all group members via GitHub Developed and edited most sections of the project proposal as the main author Added cliff mapping features to map_panel.py and connecting files for the GUI Investigated different reflective surfaces to distinguish our goal
Noah Gratz Helped develop the Scan library to accurately detect objects and locate the soldier. Integrated the Scan library into the GUI to help the drivers. Researched goal-detection methods—including those with absorbent tape, LEDs, and IR readings Synthesized and conducted real-time analysis of all gathered ProtoCue data to successfully reach the goal object as the primary driver for the demo.	Shawn Robinson completed cliff.c and imu.c for the project. Provided an API for other group members to dependably check for boundaries, and to cleanly change signal thresholds if necessary imu.c provided APIs for single-read, multi-read, write, and cardinal-direction access between our Cybot and the BNO055 IMU chip. Provided additional support with editing and formatting the project proposal and shot and edited the extra credit video	Shun Quinlan Worked on the development of the movement system, integrating cliff and bump detection, sound cues, LDR, IMU, and movement-status LEDs. Designed both manual and automated field-exit behaviors that safely leave the test area while still checking sensors and created the main visuals/diagrams for the project proposal. Developed and modified an additional IR sensor (IR modulator) for the project, as well as additional IR sensors for the goal object, PVC pipe.



Team Member Contributions to Final POV Statement

Final POV Statement: "ProtoCue needs to help this soldier in this compromised battlefield."

- Deraj - Provided context and empathy considerations for how military operations work
- Noah - Suggested a special IR-tape signifier for soldiers to be identified
- Lex - Pitched the idea for the rescue bot
- Shun - Provided illustrative sketches that helped develop understanding and empathy for the relevant user groups
- Adrian - Developed a computer interface that mirrored what military functionality would demand
- Shawn - Offered the idea of the robot being used as a signal for other army commands



Figure 10: SB-PG4 after a successful demo.



10. Conclusion

ProtoCue evolved from a conceptual reconnaissance platform into a fully integrated embedded system capable of real-time hazard detection, target identification, and mission execution within constrained field conditions. The project required careful coordination between hardware integration, sensor calibration, interrupt-driven control, and communication interfaces. Through iterative testing and refinement, particularly the transition from LDR-based detection to a directional IR modulation system. The platform achieved reliable, repeatable performance across varied environmental conditions.

The first final demonstration yielded an average performance score, revealing limitations in detection precision and in operator coordination under time pressure. Rather than viewing this outcome as a setback, it became a critical point of evaluation. Subsequent refinements included narrowing the IR detection cone, optimizing threshold behavior, and improving real-time interpretation of GUI telemetry data. Additionally, structured driver training enhanced situational awareness and decision-making speed during navigation of unpredictable field layouts.

During the second final demonstration, these technical and operational improvements resulted in one of the highest scores in the course. The improvement reflected not only higher detection accuracy and fewer false positives, but also stronger synchronization between embedded system outputs, graphical feedback, and operator control. Each field configuration presented unique spatial challenges, requiring rapid interpretation of sensor readings and adaptive movement strategies in conditions in which ProtoCue demonstrated consistent, stable performance.

Beyond meeting functional objectives, ProtoCue illustrates the importance of iterative engineering, modular system design, and real-time embedded control. The project reinforced key principles of embedded systems development: deterministic timing, hardware-software co-design, signal filtering, and system-level troubleshooting. Ultimately, ProtoCue demonstrates how thoughtful integration, disciplined testing, and adaptive problem-solving transform a prototype into a refined and high-performing embedded platform.



Works Cited

- [1] D. Prochazka, V. Šedivcová, J. Melichar, and A. Tesař, “Autonomous ground vehicles for MEDEVAC: capability assessment based on agent-based modelling,” ResearchGate, https://www.researchgate.net/publication/394488881_Autonomous_ground_vehicles_for_MEDEVAC_capability_assessment_based_on_agent-based_modelling (accessed Nov. 6, 2025).
- [2] “Casualty evacuation,” Wikipedia, https://en.wikipedia.org/wiki/Casualty_evacuation (accessed Nov. 6, 2025).
- [3] “Absolute and relative morbidity burdens attributable to various illnesses and injuries among active component members of the U.S. Armed Forces, 2024,” Military Health System, <https://health.mil/News/Articles/2025/09/01/MSMR-Health-Care-Burden-Active-Component-2024> (accessed Nov. 6, 2025).
- [4] “Photo Resistor Sensor Module Pinout | LDR,” *startingelectronics.org*. <https://startingelectronics.org/pinout/photo-resistor/>
- [5] A. Rathore, “What Is a Light-Dependent Resistor (LDR)? | Basics,” *Electronics For You*, Oct. 15, 2020. <https://www.electronicsforu.com/technology-trends/learn-electronics/ldr-light-dependent-resistors-basics>
- [6] S. Harris, “Inertial Measurement Unit (IMU) - an introduction,” *Advanced Navigation*, Feb. 12, 2023. <https://www.advancednavigation.com/tech-articles/inertial-measurement-unit-imu-an-introduction>
- [7] L. Barber, “IR Receiver and Transmitter Modules with Arduino - Meganano,” *Meganano - Simplifying tech for you*, Mar. 12, 2024. <https://meganano.uno/ir-receiver-and-transmitter/> (accessed Dec. 10, 2025).